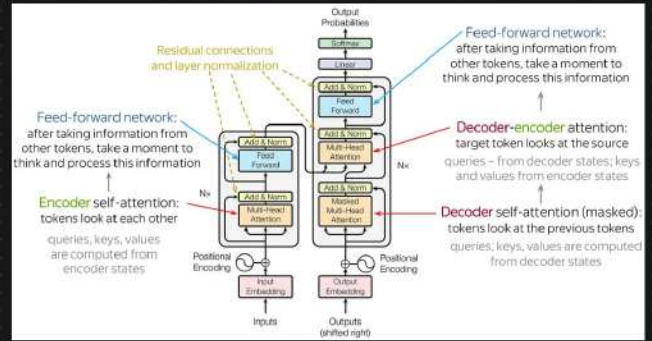


Introduction To Transformers

- 1) RNN/LSTM/GRU RNN
- 2) Encoder Decoder Architecture
- 3) ATTENTION MECHANISM
- 4) TRANSFORMERS

- ① Why Transformers?
- ② Architecture of Transformers?
- ③ SELF ATTENTION $\rightarrow Q, K, V$
- ④ Positional Encoding
- ⑤ Multi Head ATTENTION
- ⑥ Combining the working of Transformers

Architecture



Generative AI $\rightarrow$ LM, Multimodal

BERT, GPT $\leftarrow$



Open AI $\rightarrow$ Chat GPT

$\downarrow$
GPT-4D

① What And Why $\rightarrow$ Transformers

Transformers in natural language processing (NLP) are a type of deep learning model that use self-attention mechanisms to analyze and process natural language data. They are encoder-decoder models that can be used for many applications, including machine translation. $\Rightarrow$ Seq2Seq Task

Eg: Language Translation $\rightarrow$ Google Translation

English $\rightarrow$ French

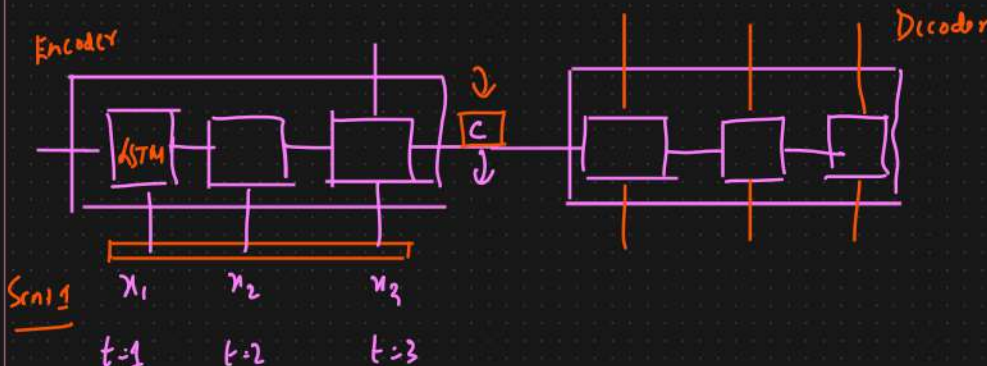
inp $\Rightarrow$ Many $\rightarrow$ o/p: Many $\cdot$ {length of the sentence}

Encoder - Decoder

Sentence length $\uparrow \uparrow$

Bleau Score $\downarrow \downarrow$

length sentence $\uparrow \uparrow$



3.1 DECODER: GENERAL DESCRIPTION

In a new model architecture, we define each conditional probability in Eq. (2) as:

$$p(y_i | y_1, \dots, y_{i-1}, x) = g(y_{i-1}, s_i, c_i), \quad (4)$$

where s_i is an RNN hidden state at time i , computed by

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

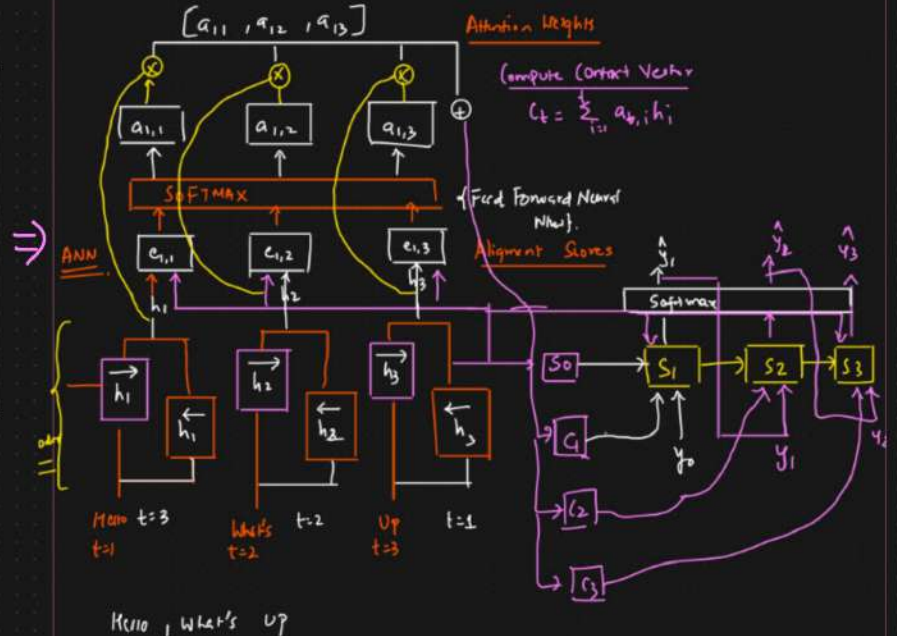
It should be noted that unlike the existing encoder-decoder approach (see Eq. (2)), here the probability is conditioned on a distinct context vector c_i for each target word y_i .

The context vector c_i depends on a sequence of annotations $(h_1, \dots, h_T)$ to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i -th word of the input sequence. We explain in detail how the annotations are computed in the next section.

The context vector c_i is, then, computed as a weighted sum of these annotations h_j :

$$c_i = \sum_{j=1}^T \alpha_{ij} h_j. \quad (5)$$

Figure 1: The graphical illustration of the proposed model trying to generate the i -th target word y_i given a source sentence $(x_1, x_2, \dots, x_T)$.



Additional Context $\rightarrow$ Decoder

long sentence. Accuracy $\uparrow\uparrow$.

Attention Mechanism

① Parallely We cannot send all the words in a sentence $\rightarrow$ Scalable

DATASET $\rightarrow$ Huge $\rightarrow$ Scalable With Respect to Training.

TRANSFORMERS $\neq$ LSTM RNN

$\hookrightarrow$ Self Attention Module $\leftarrow$ All the words will be parallelly sent to encoder.

$\Downarrow$

Positional Encoding

Transformer $\uparrow\uparrow$ DATASET $\rightarrow$ Amazing SOTA $\leftarrow$ NLP

Transfer Learning $\rightarrow$ MultiModel Task $\rightarrow$ NLP + Image $\leftarrow$

Transformers $\div$ AI Space $\rightarrow$ SOTA Model $\rightarrow$

BERT GPT $\rightarrow$ Transfer Learning $\rightarrow$ SOTA Models $\rightarrow$ DALL-E } GAN's
Generative AI

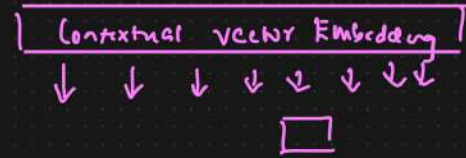
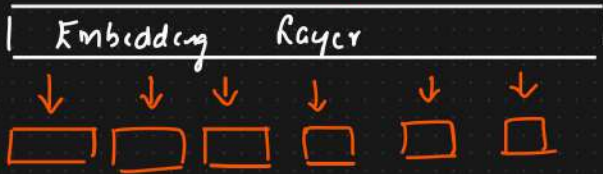
Train Huge Data

② Contextual Embedding → Self Attention

Contextual Vector

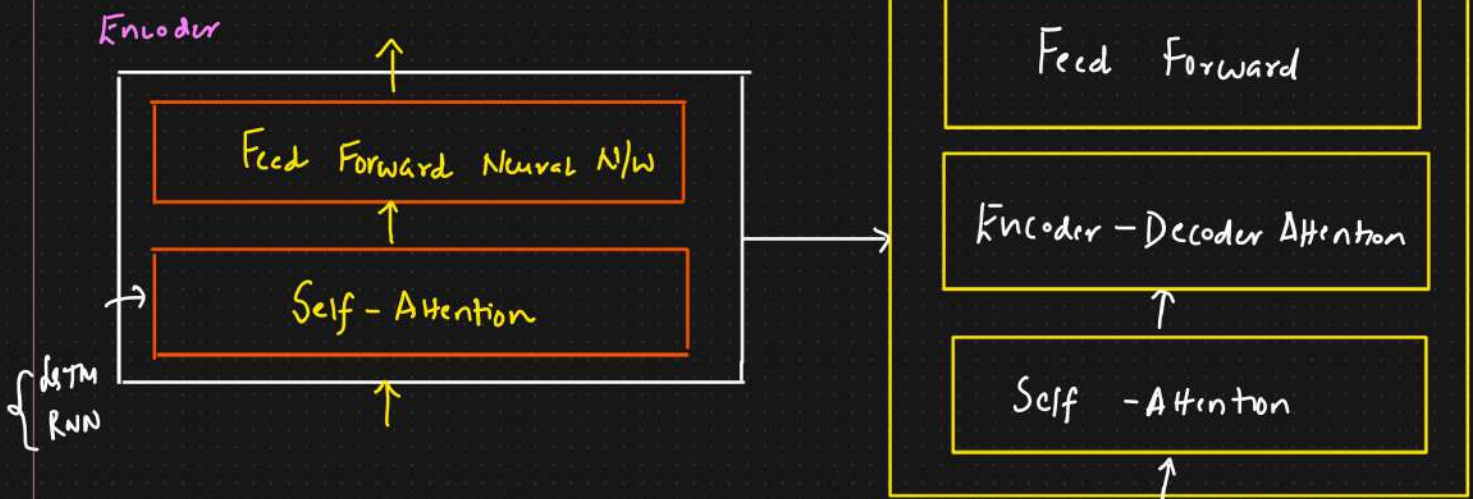
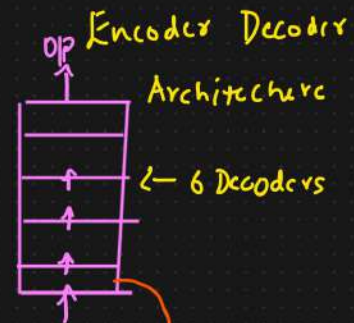
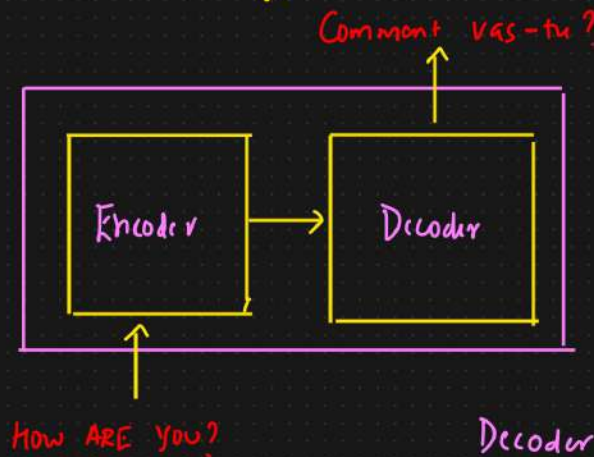
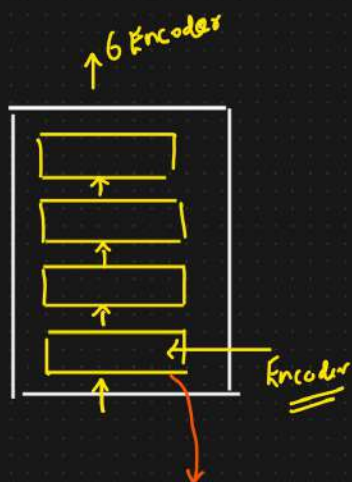
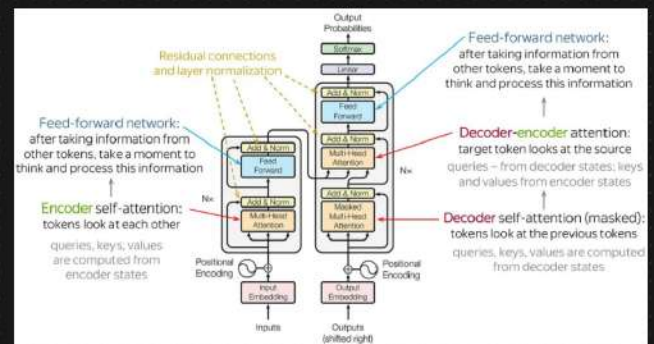
Eg: My name is KRISH And I play CRICKET

Word2Vec

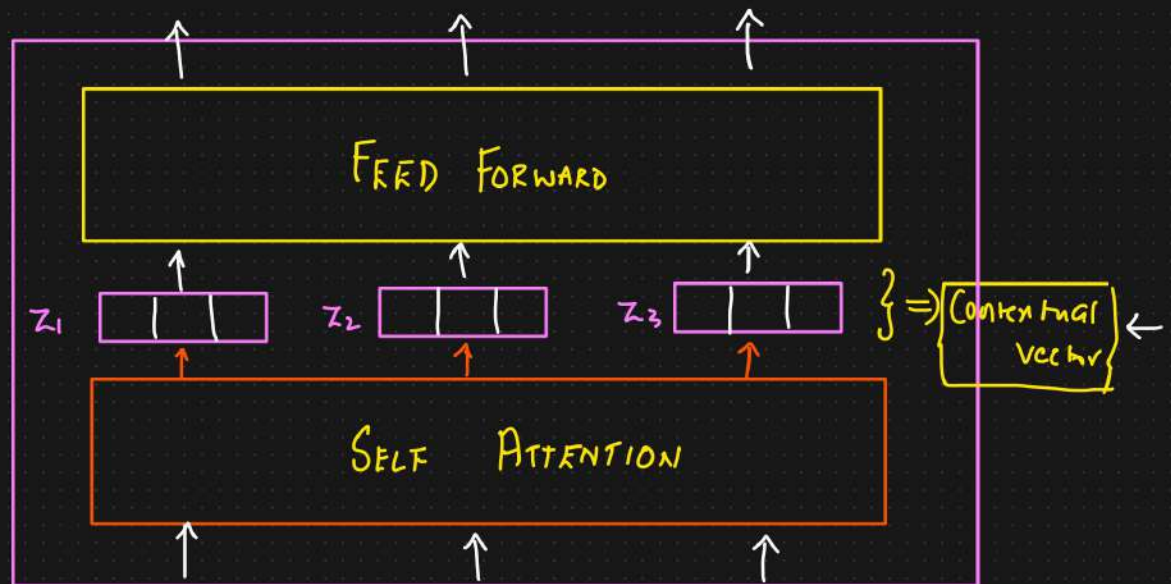


② Basic Transformer Architecture

{Seq2Seq Task} → Language Translation {Eng → French}



Encoder



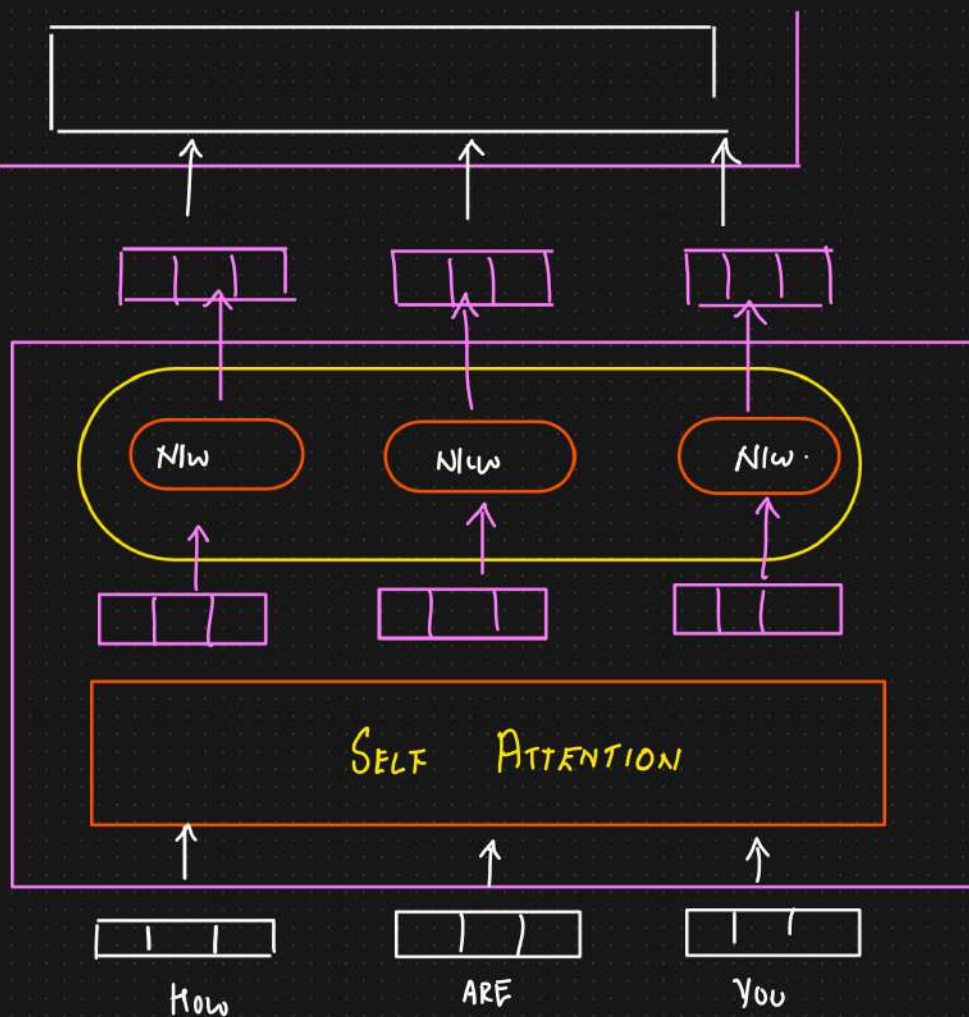
Vectors
↓
Embedding Layer



{ All the words
will be passed
parallelly }

Encoder 2

Encoder



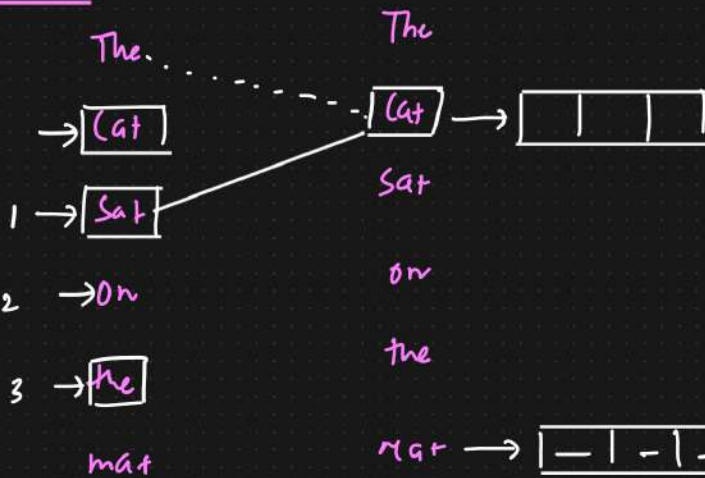
Self Attention At a Higher Level

Eg: The cat sat on the mat, the cat lay on the rug.

Word Embedding



SELF ATTENTION



{ Contextual Embedding }

Rank
1
2
3



Self Attention In Detail

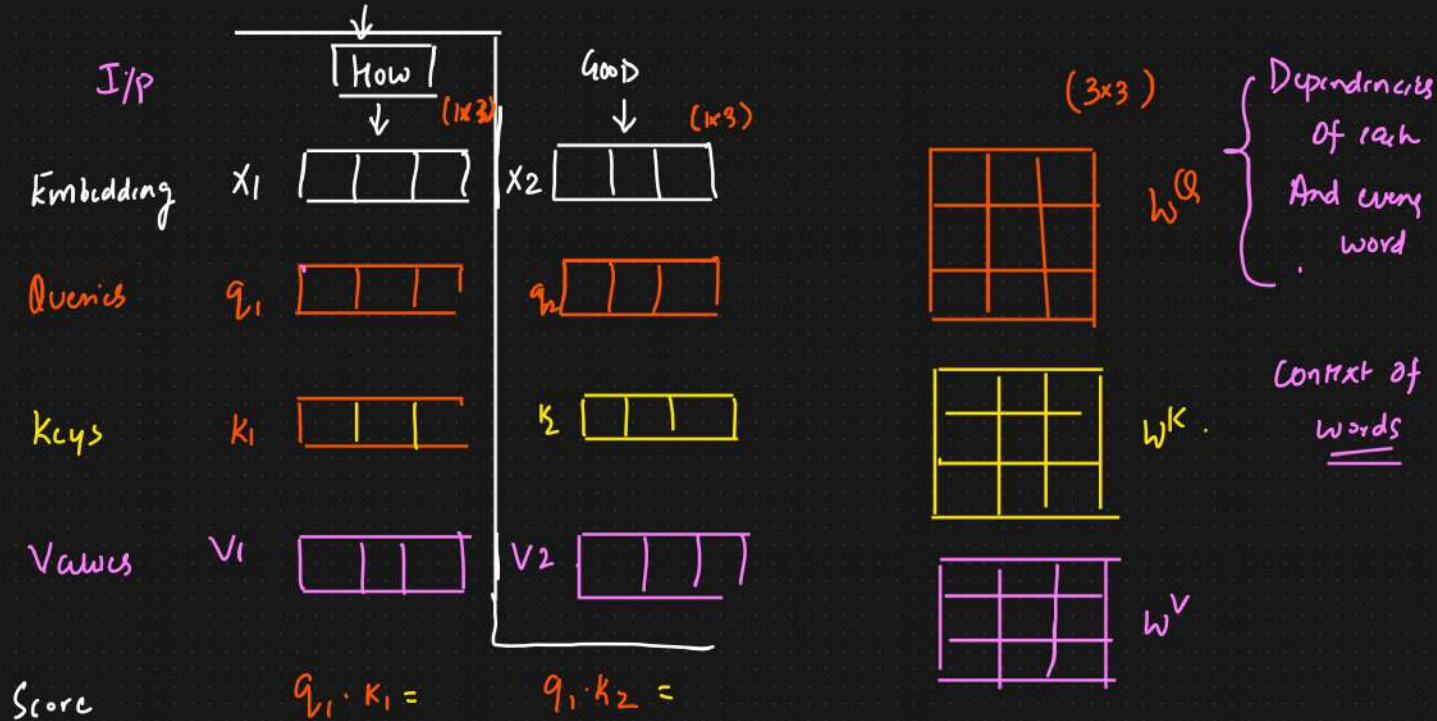
① To create 3 vectors from each of the encoder i/p: Query vector, Key vector, value vector. ⇒ Contextual Embedding

Eg: YT ⇒ Search keywords ⇒ {Query}

Query → {Key} → ^{Tags}
Description ⇒ Value ⇒ Opp Video
Title

② Second step in calculating self attention is to calculate the score.

The score determines how much focus to place on the other part of the sentence.



⊛ How much Focus to place on other part of the i/p Sentence as we encode a word at certain position

Divide $\sqrt{dk}$

↓

{ More stable }
gradients

Dimension=3

Dimension $\uparrow \uparrow \uparrow \approx 12$

O/p $\uparrow \uparrow \uparrow$

Variance $\uparrow \uparrow \uparrow \Rightarrow$

$\frac{\text{Variance}}{\sqrt{dk}} \approx \frac{\text{Variance}}{\sqrt{dk}}$

Soft+Max

$[0.88]$ $[0.12]$

$0.88 + 0.12 = 1$

Softmax $\Rightarrow$ The Softmax score determines how much each word will be expressed at this position

X

Value Vectors v_1 v_2

0.88 $[1 \ 1 \ 1]$

$[0.88 \ 0.88 \ 0.88]$ + $[0.12 \ 0.12 \ 0.12]$

Contextual vector

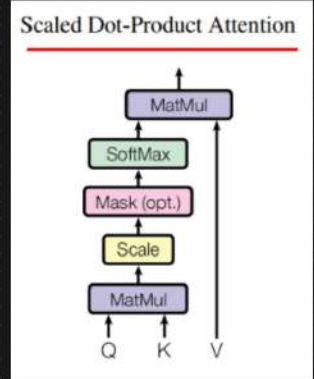
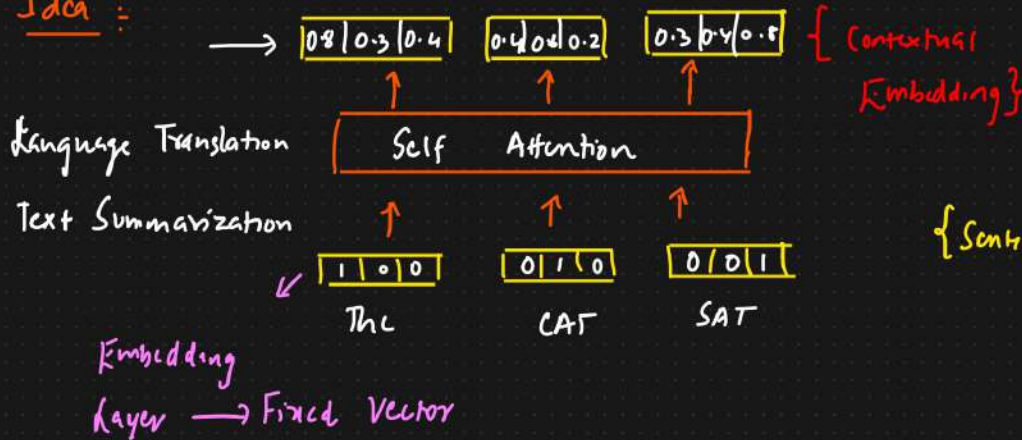
z_1 $[1 \ 1 \ 1]$

z_2 $[1 \ 1 \ 1]$

Self Attention At Higher And Detailed Level

Self-attention, also known as scaled dot-product attention, is a crucial mechanism in the transformer architecture that allows the model to weigh the importance of different tokens in the input sequence relative to each other

Idea :



1) Inputs: Queries, Keys, And Values

Model → Queries, Keys And Values

1. Query Vectors (Q):

Role: Query vectors represent the token for which we are calculating the attention. They help determine the importance of other tokens in the context of the current token.

Importance:

Focus Determination: Queries help the model decide which parts of the sequence to focus on for each specific token. By calculating the dot product between a query vector and all key vectors, the model assesses how much attention to give to each token relative to the current token.

Contextual Understanding: Queries contribute to understanding the relationship between the current token and the rest of the sequence, which is essential for capturing dependencies and context.

2. Key Vectors (K):

Role: Key vectors represent all the tokens in the sequence and are used to compare with the query vectors to calculate attention scores.

Importance:

Relevance Measurement: Keys are compared with queries to measure the relevance or compatibility of each token with the current token. This comparison helps in determining how much attention each token should receive.

Information Retrieval: Keys play a critical role in retrieving the most relevant information from the sequence by providing a basis for the attention mechanism to compute similarity scores.

3. Value Vectors (V):

Role: Value vectors hold the actual information that will be aggregated to form the output of the attention mechanism.

Importance:

Information Aggregation: Values contain the data that will be weighted by the attention scores. The weighted sum of values forms the output of the self-attention mechanism, which is then passed on to the next layers in the network.

Context Preservation: By weighting the values according to the attention scores, the model preserves and aggregates relevant context from the entire sequence, which is crucial for tasks like translation, summarization, and more.

Input Sequence = ["The", "CAT", "SAT"]

Embedding size = 4

$Q, K, V \Rightarrow 4$

$\hookrightarrow$ [Self Attn] $\rightarrow$

[0.6 | 0.2 | 0.3] $\leftarrow$

[1 | 0 | 0] [] []

$\downarrow$
Sentence, Dataset

① Token Embedding

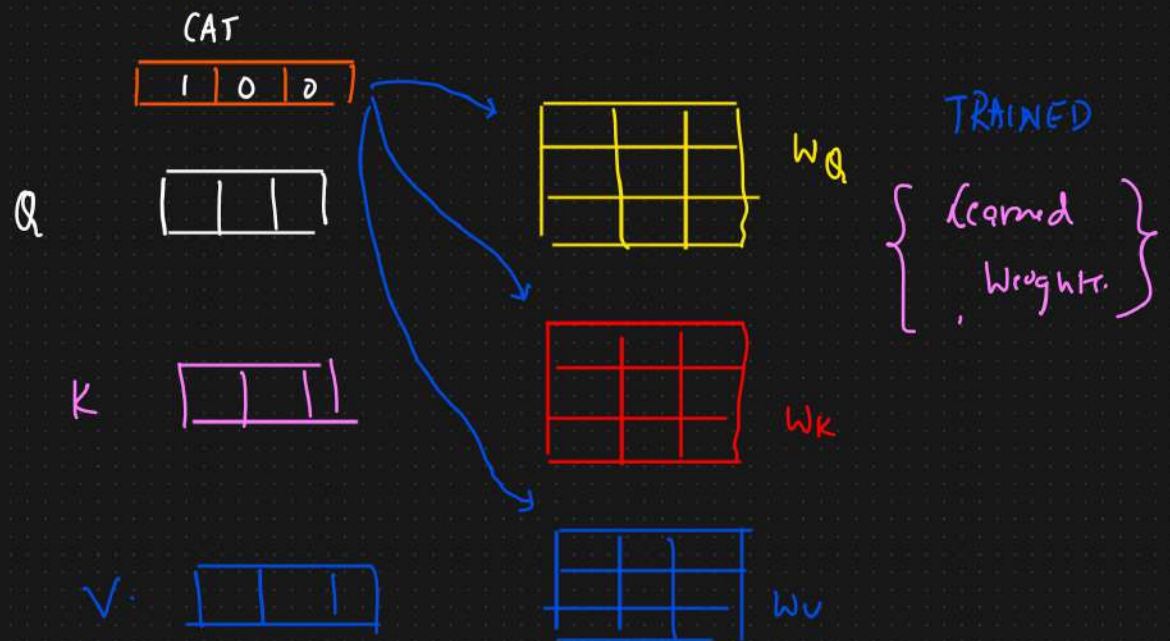
$$E_{\text{The}} = [1 \ 0 \ 1 \ 0]$$

$$E_{\text{CAT}} = [0 \ 1 \ 0 \ 1]$$

$$E_{\text{SAT}} = [1 \ 1 \ 1 \ 1]$$

② Linear Transformation

We create Q, K, V by multiplying the embeddings by learned weights matrices W_Q, W_K and W_V .



Let's consider

$$W_Q = W_K = W_V = I$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \leftarrow$$

$$Q_{The} = [1 \ 0 \ 10] \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [1 \ 0 \ 10]$$

$$K_{The} = [1 \ 0 \ 10]$$

$$V_{The} = [1 \ 0 \ 10]$$

$$① \ Q_{The} = K_{The} = V_{The} = [1 \ 0 \ 10]$$

$$② \ Q_{CAT} = K_{CAT} = V_{CAT} = [0 \ 1 \ 0 \ 1]$$

$$③ \ Q_{SAT} = K_{SAT} = V_{SAT} = [1, 1, 1, 1]$$

③ Compute Attention Scores

$$\begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

The

$$\text{Score}(Q_{\text{The}}, K_{\text{The}}) = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}^T = 2$$

$$\text{Score}(Q_{\text{The}}, K_{\text{CAT}}) = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}^T = 0$$

$$\text{Score}(Q_{\text{The}}, K_{\text{SAT}}) = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}^T = 2$$

For the token CAT

$$\text{Score}(Q_{\text{CAT}}, K_{\text{The}}) = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}^T = 0$$

$$\text{Score}(Q_{\text{CAT}}, K_{\text{CAT}}) = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}^T = 2$$

$$\text{Score}(Q_{\text{CAT}}, K_{\text{SAT}}) = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}^T = 2$$

For the Token SAT

$$\left\{ \begin{array}{l} \text{Score}(Q_{\text{SAT}}, K_{\text{The}}) = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}^T = 2 \\ \text{Score}(Q_{\text{SAT}}, K_{\text{CAT}}) = 2 \\ \text{Score}(Q_{\text{SAT}}, K_{\text{SAT}}) = 4 \end{array} \right.$$

④ Scaling :

We take up the scores and scale down by dividing the scores by the

$$\sqrt{d_k} \Rightarrow d_k = 4 \quad \sqrt{d_k} = 2$$

Scaling in the attention mechanism is crucial to prevent the dot product from growing too large. $\Rightarrow$ Ensure stable gradients during Training

d_k is large $\rightarrow$

① Gradient Exploding

② Softmax Saturation $\left\{ \int \right\} \rightarrow$ Vanishing Gradient Problem.

$$Q = \begin{bmatrix} 2 & 3 & 4 & 1 \end{bmatrix} \quad K_1 = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix} \quad K_2 = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}$$

Without Scaling

$$Q \cdot K_1^T = 2 \times 1 + 3 \times 0 + 4 \times 1 + 1 \times 0 = 2 + 4 = 6$$

$$Q \cdot K_2^T = 2 \times 0 + 3 \times 1 + 4 \times 0 + 1 \times 1 = 0 + 3 + 0 + 1 = 4$$

* Score $[6, 4] \Rightarrow$ Scaling Not Applied

$$\text{Softmax}([6, 4]) = \left[\frac{e^6}{e^6 + e^4}, \frac{e^4}{e^6 + e^4} \right] = \left[\frac{e^6}{e^6(1 + e^{-2})}, \frac{e^4}{e^4(e^2 + 1)} \right]$$

④ Property of Softmax

W_Q, W_K, W_V

$$([10, 1]) = [0.99, 0.01]$$

Dot product = Large values

$$= \left[\frac{1}{(1 + e^{-2})}, \frac{1}{(e^2 + 1)} \right]$$

$$\approx [0.88, 0.12]$$

Most of the attention weight is assigned to the first key vector, very little to the second vector,

With Scaling

$$\sqrt{d_k} = \sqrt{4} = \sqrt{2} \Rightarrow \text{Variance } 2, 3$$

① Compute Scaled Dot Product

$$[6, 4] \Rightarrow \text{Scale} \Rightarrow \left[\frac{6}{2}, \frac{4}{2} \right] = [3, 2]$$

$$\text{Softmax}([3, 2]) = \left[\frac{e^3}{e^3 + e^2}, \frac{e^2}{e^3 + e^2} \right] = \left[\frac{e^3}{e^3(1 + e^{-1})}, \frac{e^2}{e^2(e + 1)} \right]$$

$$= [0.73, 0.27] \Rightarrow \text{Attention Weights}$$

(*) Here, the attention weights are more balanced compared to the unscaled case.

Summary of Importance

Stabilizing Training: Scaling prevents extremely large dot products, which helps in stabilizing the gradients during backpropagation, making the training process more stable and efficient.

Preventing Saturation: By scaling the dot products, the softmax function produces more balanced attention weights, preventing the model from focusing too heavily on a single token and ignoring others.

Improved Learning: Balanced attention weights enable the model to learn better representations by considering multiple relevant tokens in the sequence, leading to better performance on tasks that require context understanding.

Scaling ensures that the dot products are kept within a range that allows the softmax function to operate effectively, providing a more balanced distribution of attention weights and improving the overall learning process of the model.

(4) Scaling $= \sqrt{d_k} = \sqrt{4} \Rightarrow 2$

$$\text{Scaled-Score} (Q_{The}, K_{The}) = \frac{2}{2} = 1$$

$$\text{Scaled-Score} (Q_{The}, K_{CAT}) = \frac{0}{2} = 0$$

$$\text{Scaled-Score} (Q_{The}, K_{SAT}) = \frac{2}{2} = 1$$

Similarly scaling
will be done for
all other Tokens.

(5) Apply Softmax

$$\text{ATTENTION WEIGHTS}_{"The"} = \text{Softmax}([1, 0, 1]) = [0.4223, 0.1554, 0.4223]$$

$$\text{ATTENTION WEIGHTS}_{"CAT"} = \text{Softmax}([0, 2, 2]) = [0.1554, 0.4223, 0.4223]$$

$$\text{ATTENTION WEIGHTS}_{"SAT"} = \text{Softmax}([2, 2, 4]) = [0.2119, 0.2119, 0.5762]$$

(6) Weight Sum of Values

We multiply the attention weights by corresponding value vectors

For the Token The =

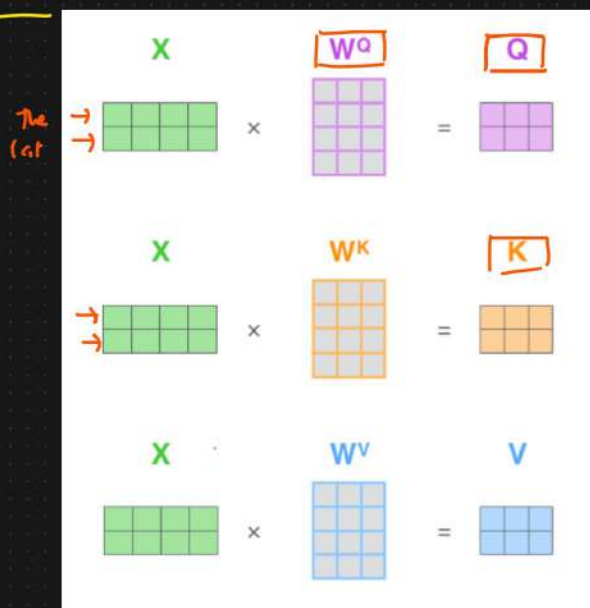
$$\begin{aligned}
 \text{Output}_{(\text{the})} &= 0.4223 * V_{\text{The}} + 0.1554 * V_{\text{cat}} + 0.4223 * V_{\text{sat}} \\
 &= 0.4223 [1 \ 0 \ 1 \ 0] + 0.1554 [0 \ 1 \ 0 \ 1] + 0.4223 [1 \ 1 \ 1 \ 1] \\
 &= [0.4223, 0, 0.4223, 0] + [0, 0.1554, 0, 0.1554] + [0.4223, 0.4223, 0.4223, 0.4223] \\
 &= [1.2669, 0.9999, 1.2669, 0.9999]
 \end{aligned}$$

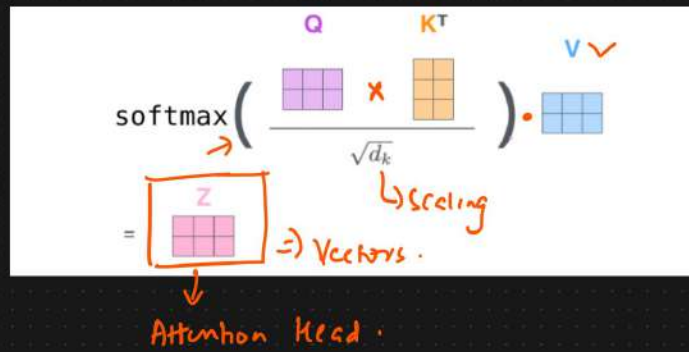
Contextual vector

The $[1 \ 0 \ 1 \ 0] \Rightarrow \text{Self Attention} \Rightarrow [1.2669, 0.9999, 1.2669, 0.9999]$

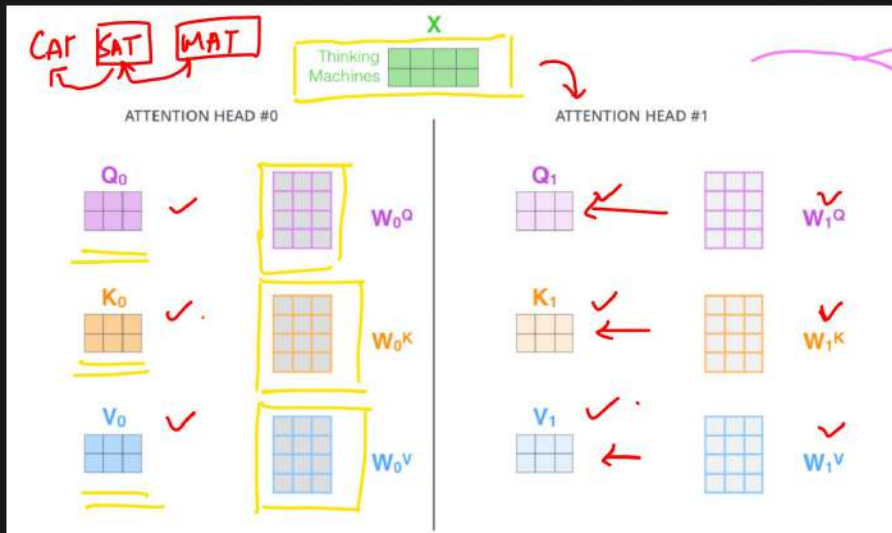
- ① $\hookrightarrow Q, K, V [W^Q, W^K, W^V]$
- ② $\hookrightarrow \text{Attention Score}$
- ③ $\hookrightarrow \text{Scaled}$
- ④ $\hookrightarrow \text{Softmax}$
- ⑤ $\hookrightarrow \text{Weighted Sum of Value (Softmax} \times V)$

④ Multi Head Attention





→ Self Attention with Multi Heads



It expands model's ability to focus on different position of tokens.

Score, Softmax $X V$

↓

$[\text{---}]$

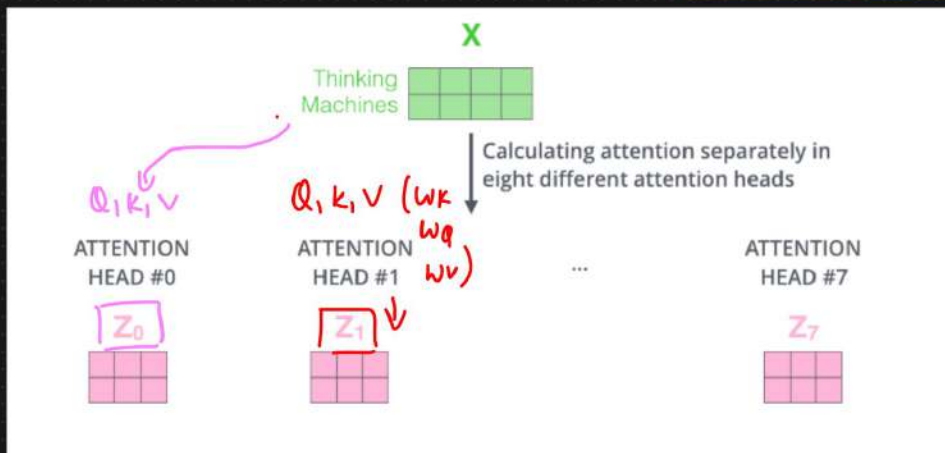
Z_0

Vectors

$[\text{---}]$

Z_1

Multi Head Attention

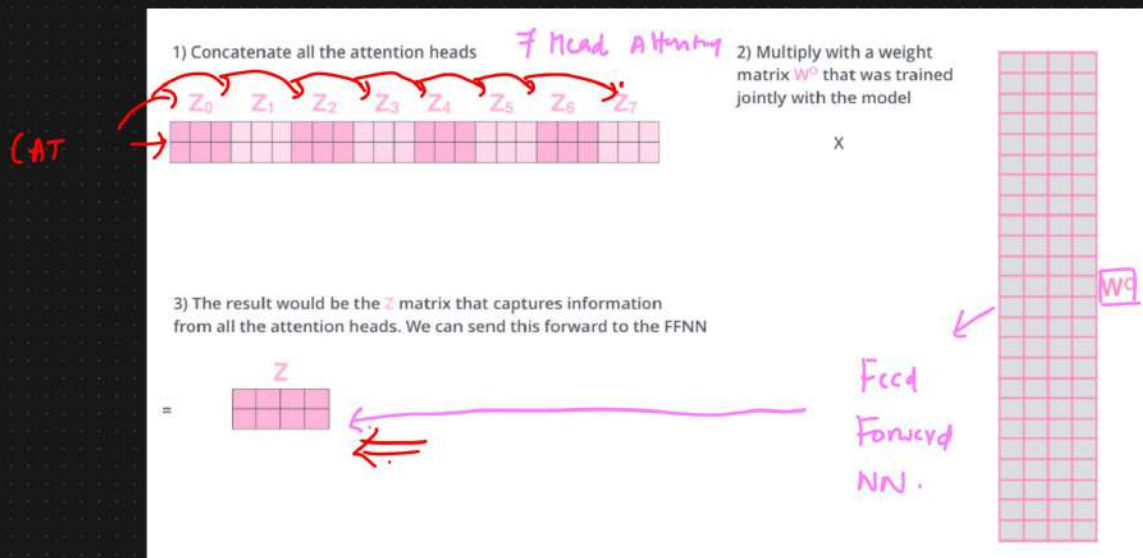
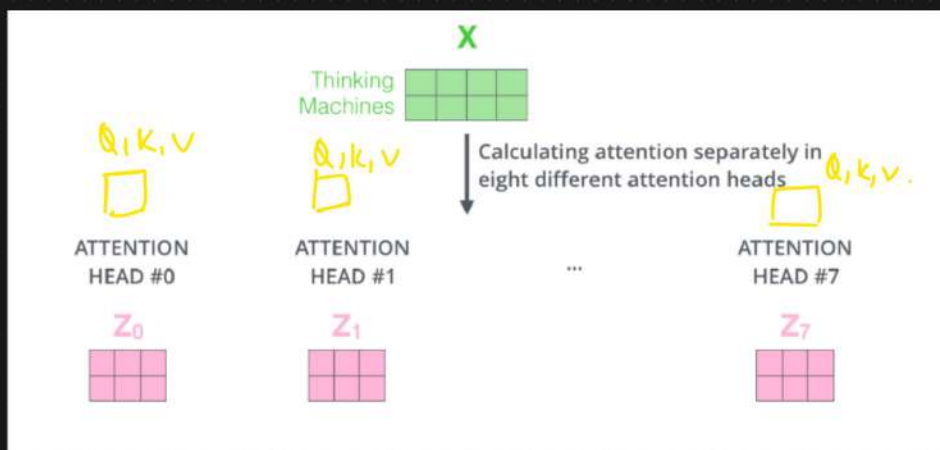
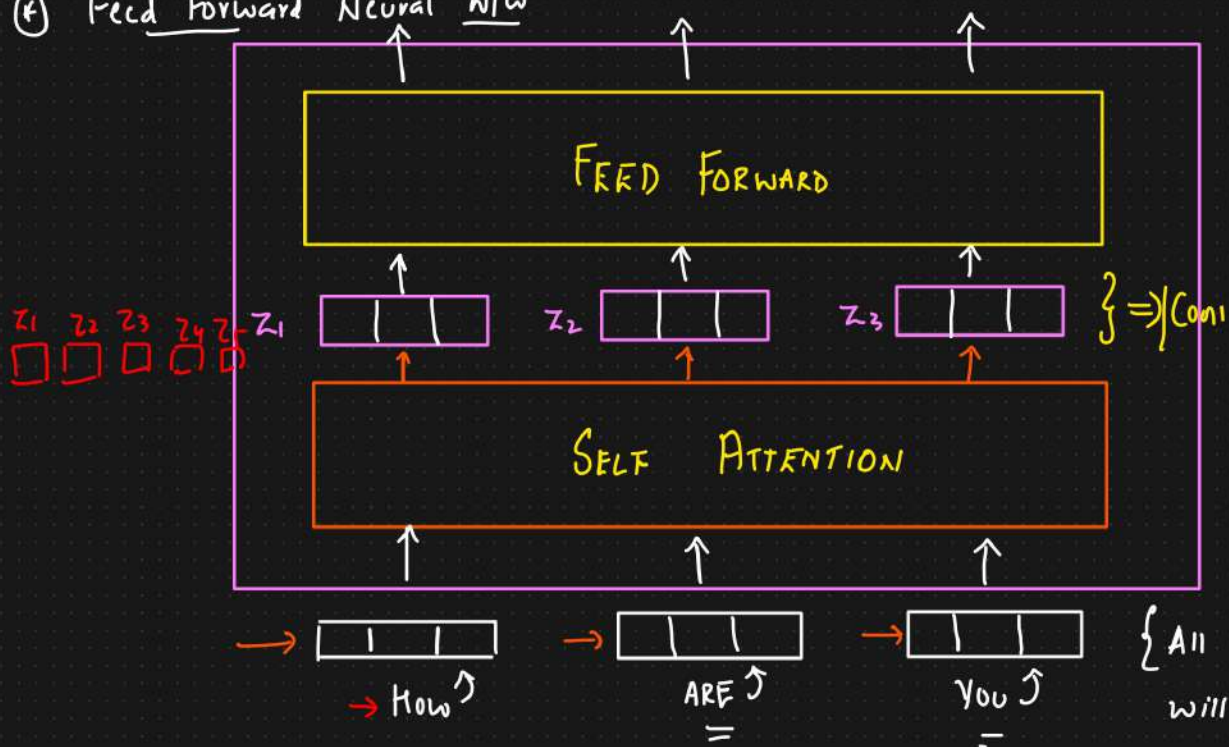


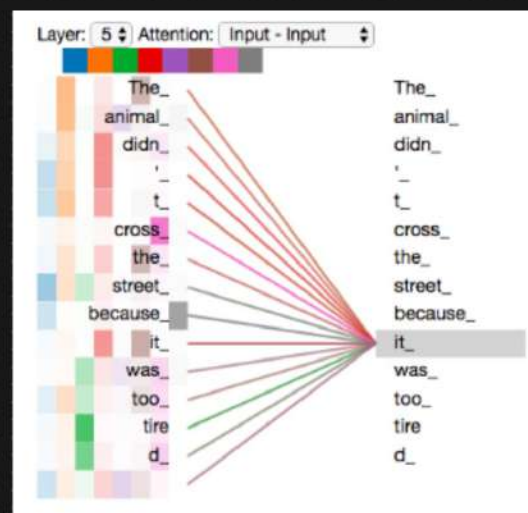
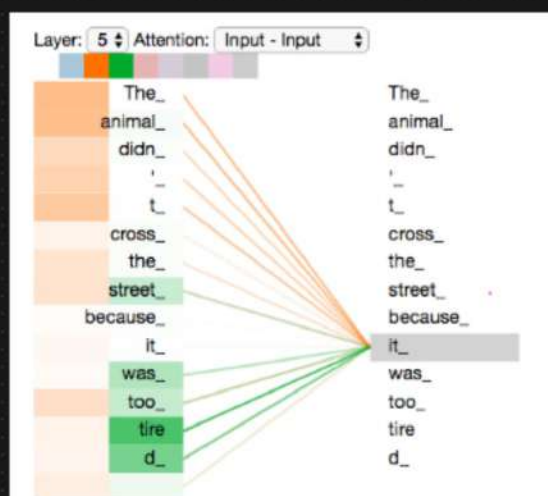
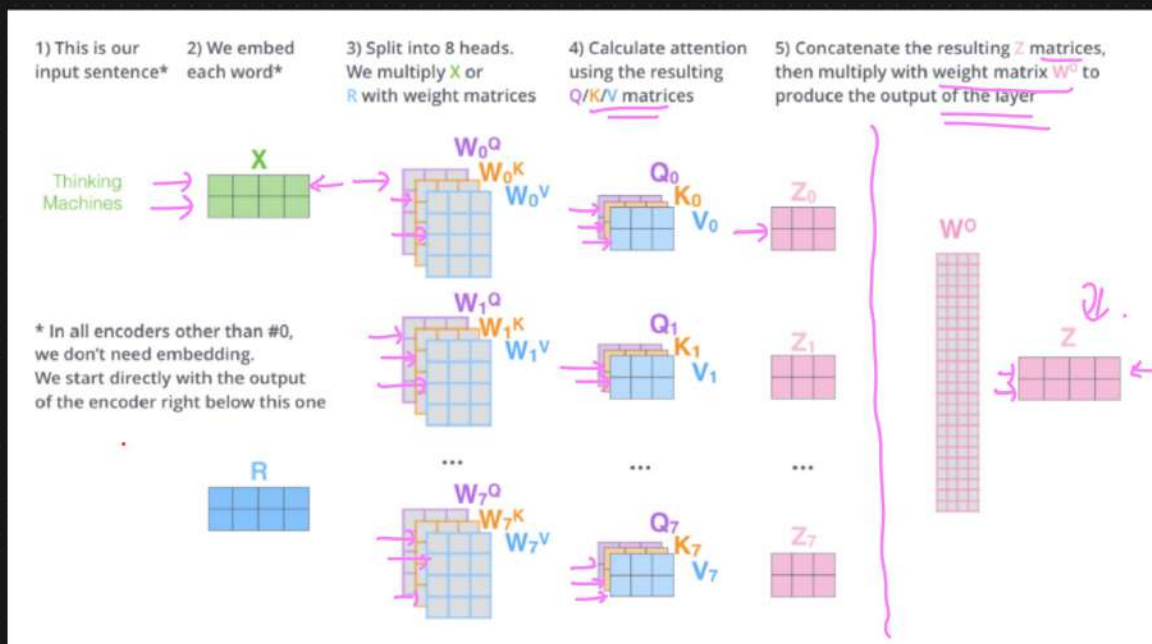
Z

Z

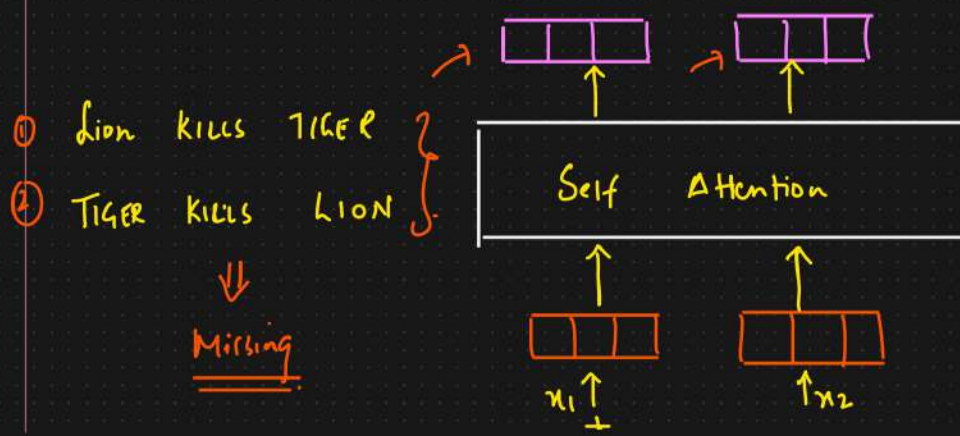
Z

④ Feed Forward Neural NW





① Positional Encoding - Representing Order of Sequence



Advantage

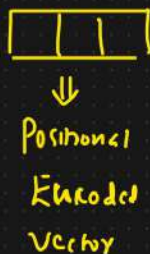
- ① Word Tokens it can process parallelly



DRAWBACK

Lack the sequential structure of the words {order}

Attention IS ALL
YOU NEED



Book, Journal
Novel

↳ 1 lakh words

↓
Backpropagation



Types of Position Encoding

✓ 1) Sinusoidal Position Encoding →

2) Learned Positional Encoding ⇒ Positional Encoding Are learned during Training ←

① Sinusoidal Positional Encoding : It uses sine and cosine functions of different frequencies to create positional encodings

Formula :



$$P.E(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$P.E(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

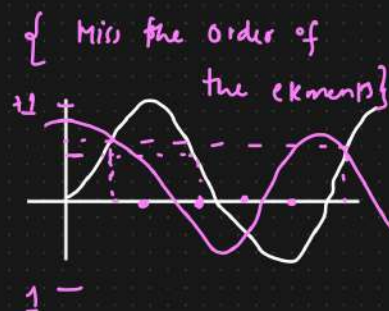
Where pos is the position
i is the dimension
d_{model} is the dimensionality of the embeddings.

Eg: The Cat Sat

The → [0.1 0.2 0.3 0.4]

CAT → [0.5 0.6 0.7 0.8]

SAT → [0.9 1.0 1.1 1.2]



$$P.E(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$P.E(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For our example $d_{model} = 4$

For position $pos = 0$

$$P.E(0, 0) = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0$$

$$P.E(0, 1) = \cos\left(\frac{0}{10000^{1/4}}\right) = \cos(0) = 1$$

$$P.E(0, 2) = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0$$

$$P.E(0, 3) = \cos\left(\frac{0}{10000^{3/4}}\right) = 1$$

$$P.E = [0, 1, 0, 1]$$

$$P.E(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For $pos = 1$

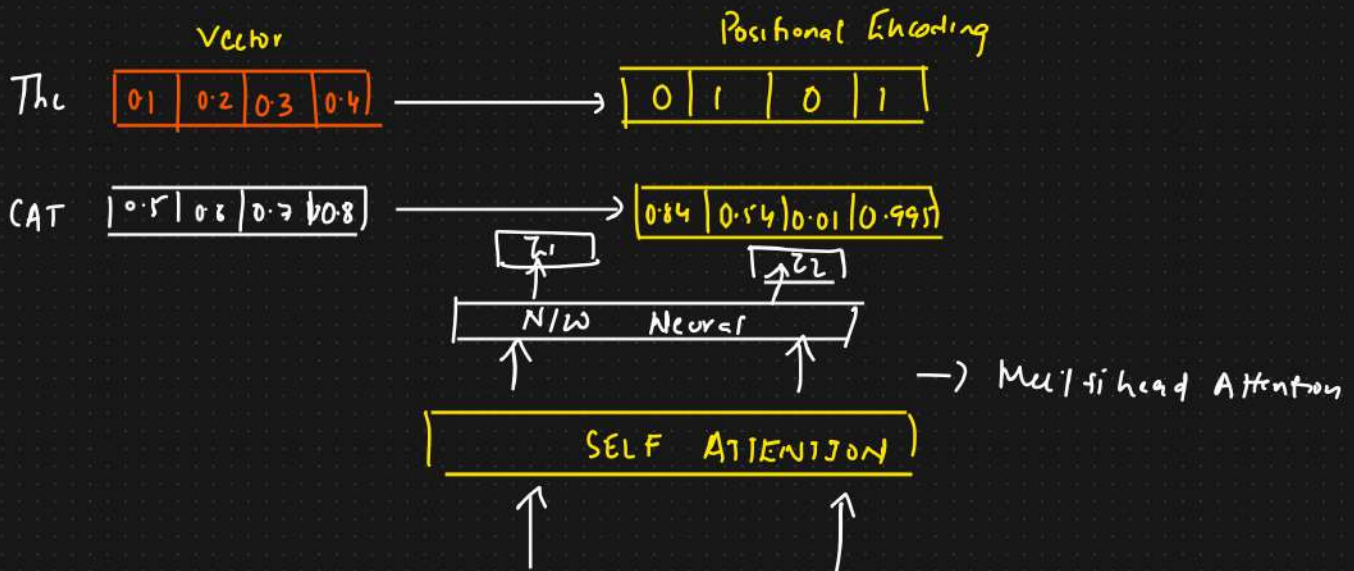
$$P.E(1, 0) = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) = 0.8415$$

$$P.E(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

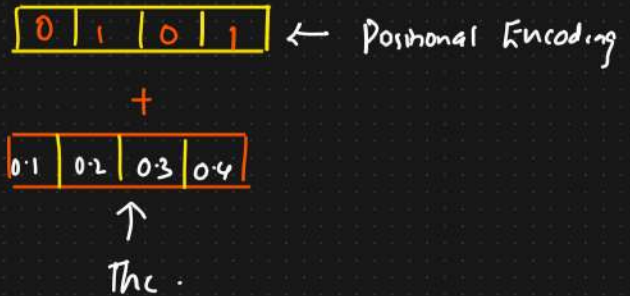
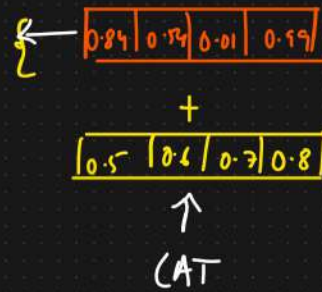
$$P.E(1, 1) = \cos\left(\frac{1}{10000^{1/4}}\right) \approx 0.5403$$

$$P.E(1, 2) = \sin\left(\frac{1}{10000^{2/4}}\right) \approx 0.01$$

$$P.E(1, 3) = \cos\left(\frac{1}{10000^{3/4}}\right) \approx 0.99995$$



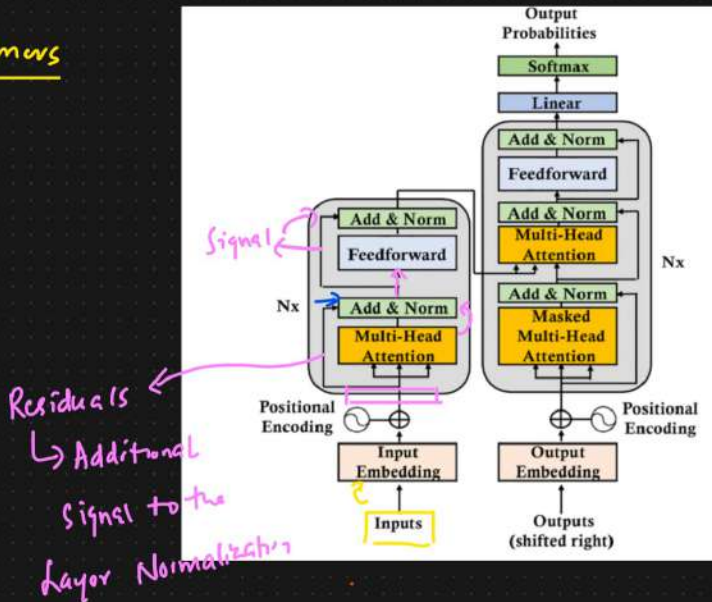
Positional
Encoding



① Layer Normalization In Transformers

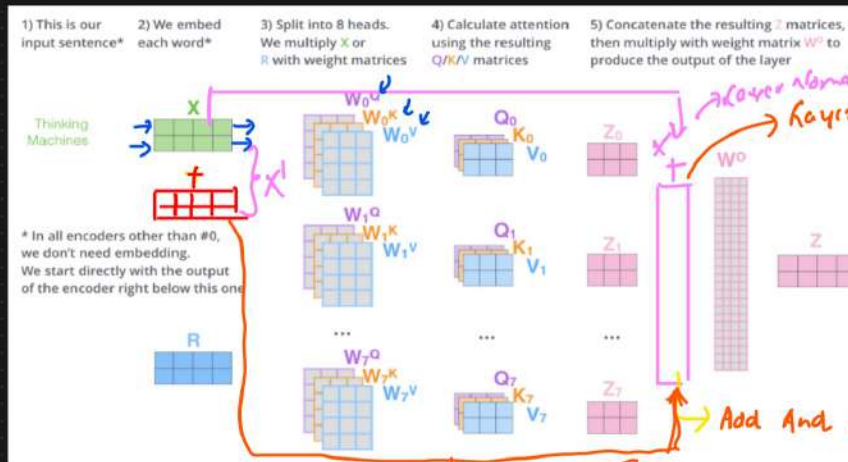
Transformers

Residuals



- ① Self Attention layer
- ② Multi head Attention
- ③ Positional Encoding
- ④ Layer Normalization

ADD AND Normalize



Normalization

Batch Normalization

Layer Normalization



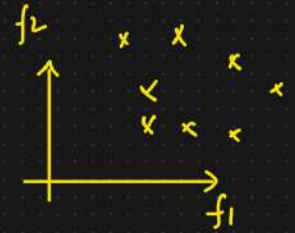
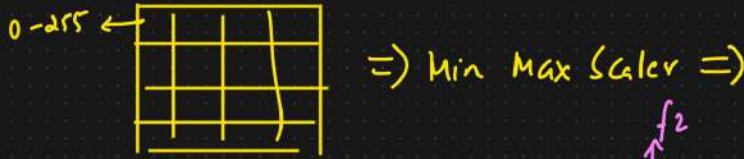
f_1	f_2	Price
House Size	No. of Rooms	
1200	2	45
1500	3	70

Normalization: Standard Scaling

$\mu, \sigma \leftarrow \begin{vmatrix} 2000 \\ 3.5 \end{vmatrix} \quad 3.5 \quad 80$

$z_{score} = \frac{x_i - \mu}{\sigma} \Rightarrow \boxed{\mu=0, \sigma=1} \quad f_1 \longrightarrow f_1' \leftarrow \mu=0, \sigma=1$

Deep learning: i/p Image $\Rightarrow$ Min Max Scaler



Advantages

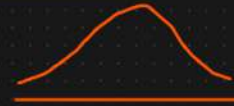
① Improved Training Stability



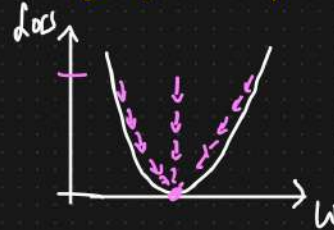
Vanishing and exploding Gradient problem



$\begin{cases} \mu=0 \\ \sigma=1 \end{cases} \quad \mu=0, \sigma=1$



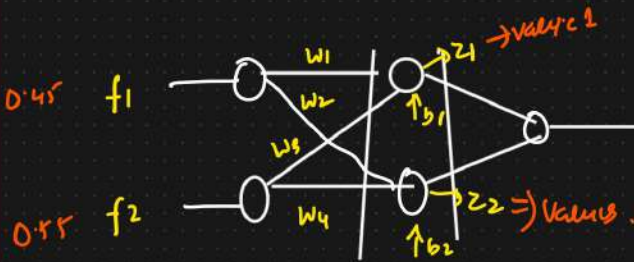
② Faster Convergence



$\Rightarrow$ Back propagation $\Rightarrow$ Stable updates.

{Batch Normalization}

Normalized Normalized



f_1 Mouse size	f_2 Rooms	Price	z_1	z_2
$\rightarrow \begin{bmatrix} 0.45 \\ 0.60 \\ - \end{bmatrix}$	$\begin{bmatrix} 0.55 \\ 0.20 \\ - \end{bmatrix}$	45	$\begin{bmatrix} - \\ - \\ - \end{bmatrix}$	$\begin{bmatrix} - \\ - \\ - \end{bmatrix}$

$z_1 = \sigma \left[(0.45 * w_1 + 0.55 * w_3) + b_1 \right] = \text{Value 1}$
 $z_2 = \sigma \left(\quad \quad \quad + b_2 \right) = \text{Value 2}$
 $\mu_1, \sigma_1 \quad \mu_2, \sigma_2$

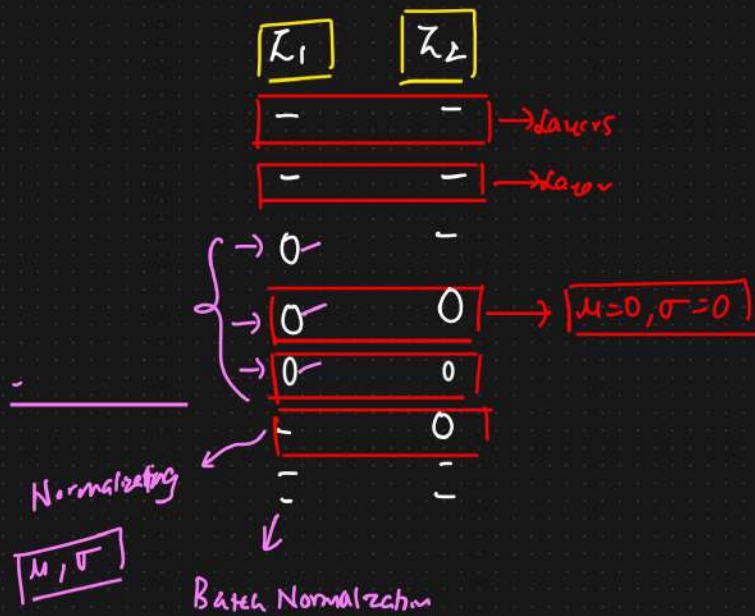
Batch Normalization VS Layer Normalization

f_1	f_2	z_1	z_2	
-	-	-	-	μ_1, σ_1
-	-	-	-	μ_2, σ_2
-	-	-	-	μ_3, σ_3

$z_{score} = \frac{x_i - \mu_1}{\sigma_1}$
 $z_{score} = \frac{x_i - \mu_2}{\sigma_2}$
 $z_{score} = \frac{x_i - \mu_3}{\sigma_3} \dots$

Layer Normalization

$\gamma, \beta \rightarrow$ learnable parameters



Normalization

γ, β

$$z_1 = \sigma [w_1^T x + b_1]$$

$$y = \gamma \left[\frac{z_1 - \mu_1}{\sigma_1} \right] + \beta$$

Scale And Shift parameters

normalized.

1) "CAT" = $[2.0, 4.0, 6.0, 8.0] \leftarrow \{\text{Values}\}$

2) Parameters = $\gamma = [1.0, 1.0, 1.0, 1.0] \rightarrow$ learned scale

$\beta = [0.0, 0.0, 0.0, 0.0] \rightarrow$ shift

$\Rightarrow$ Scale And Shift param

i) Compute the mean

$$z_{score} = \frac{x_i - \mu}{\sigma}$$

$$\mu = \frac{1}{4} (2.0 + 4.0 + 6.0 + 8.0)$$

$$= \frac{20.0}{4} = 5.0$$

ii) Compute the variance (σ^2)

$$\sigma^2 = \frac{1}{4} [(2.0 - 5.0)^2 + (4.0 - 5.0)^2 + (6.0 - 5.0)^2 + (8.0 - 5.0)^2] - 5.0$$

iii) Normalize the i/p

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$\epsilon = 1e^{-5} \Rightarrow$ Avoid division by 0

$$\sqrt{\sigma^2 + \epsilon} = \sqrt{5.0 + 1e^{-5}} \approx \sqrt{5.00001} = 2.236$$

$$\hat{x}_1 = \frac{2.0 - 5.0}{2.236} \approx -1.34$$

$$\hat{x}_2 = \frac{4.0 - 5.0}{2.236} \approx -0.45$$

$$\hat{x}_3 = \frac{6.0 - 5.0}{2.236} \approx 0.45$$

$$\hat{x}_4 = \frac{8.0 - 5.0}{2.236} \approx 1.34$$

Normalized vector

$$\hat{x} = [-1.34, -0.45, 0.45, 1.34]$$

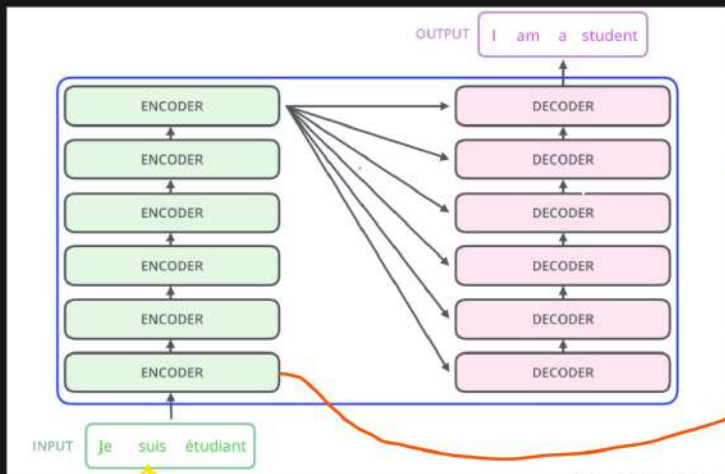
4) Scale And Shift

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

$$\gamma = [1.0, 1.0, 1.0, 1.0] \quad \beta = [0.0, 0.0, 0.0, 0.0]$$

$$y = [-1.34, -0.45, 0.45, 1.34]$$

④ Encoder Architecture [Research Paper]



Sequence to
Sequence
↓
(Complex)

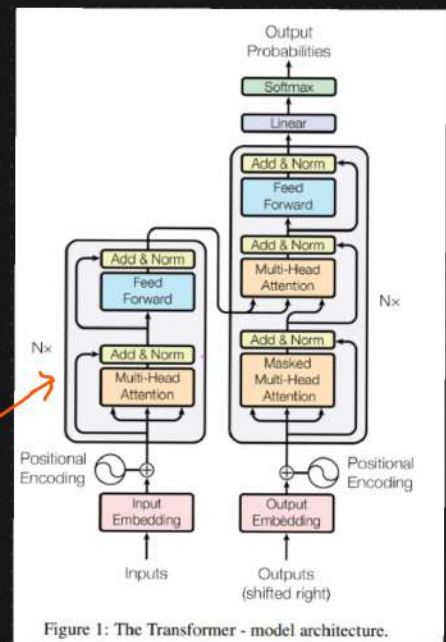
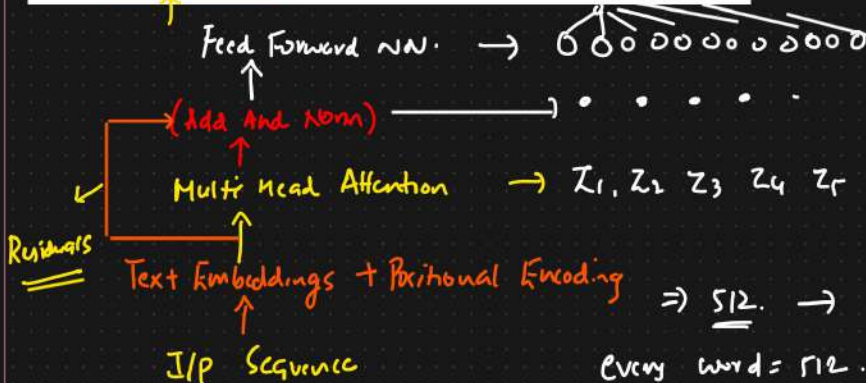


Figure 1: The Transformer - model architecture.



$$d_k = 64$$

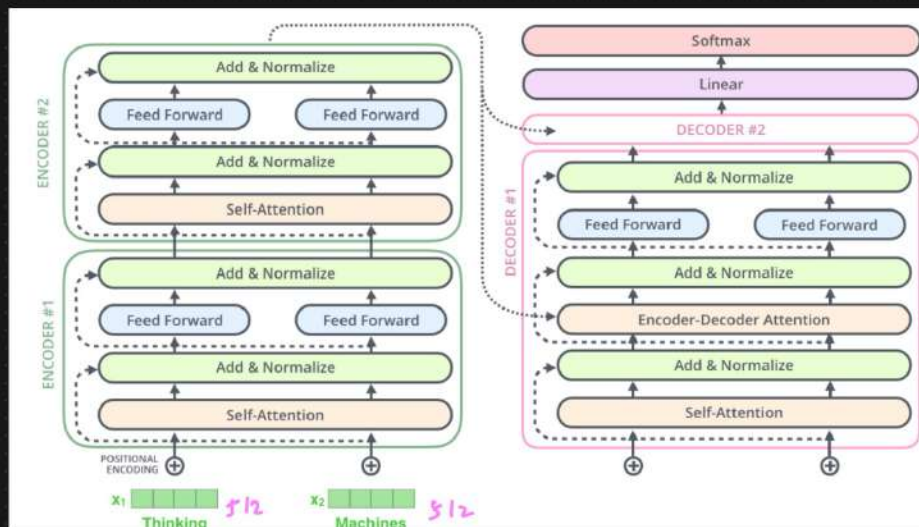
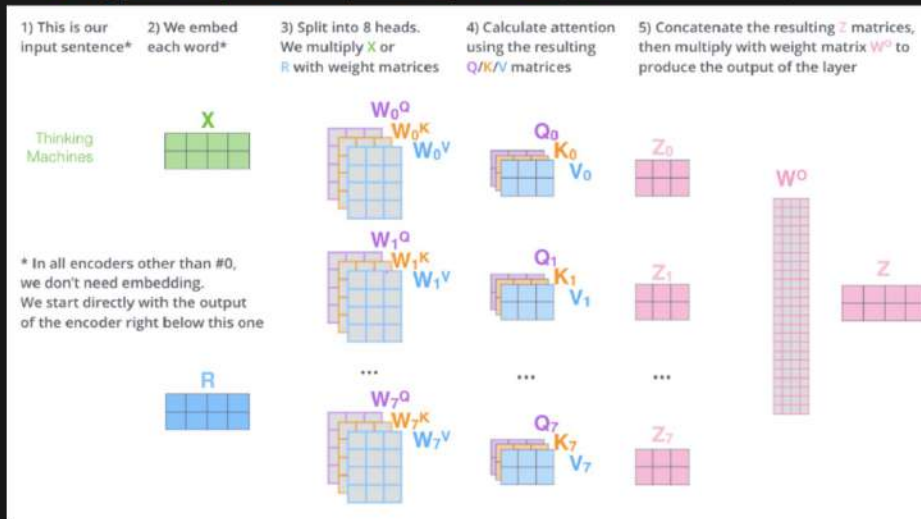
$$K = 64$$

$$V = 64$$

$$\sqrt{64} = 8$$

Every word = 512.

Self Attention to Feed Forward Neural N/w



⑥ Residual connection : Skip connection NN.

1) Addressing the Vanishing Gradient Problem

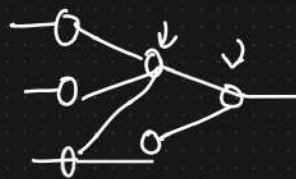
Residuals : Residual connection create a short paths for gradients to flow directly through the n/w. Gradient remains sufficiently large.

2) Improve Gradient flow

Convergence will be faster.

3) Enables Training of Deeper Networks.

④ Feed Forward NN



Linear function problem
{Non linear functions}

① Adding Non linearity

② Processing Each position Independently.

Self Attention $\rightarrow$ (capture relationships)

FFN $\rightarrow$ Each token representation Independently.



Transforming these representation further
and allows the model to learn
Richer Representation.

ANW $\Rightarrow$

③ FFN $\rightarrow$ Deeper $\Rightarrow$ Adds Depth to the Model.

Depth $\uparrow \uparrow \Rightarrow$ More learnings $\rightarrow$ DATA

④ Decoders In Transformers

3 main Components

The transformer decoder is responsible for generating the output sequence one token at a time, using the encoder's output and the previously generated tokens.

① Masked Multi Head Self Attention ✓

② Multi Head Attention (Encoder Decoder Attention)

③ Feed Forward Neural Network.

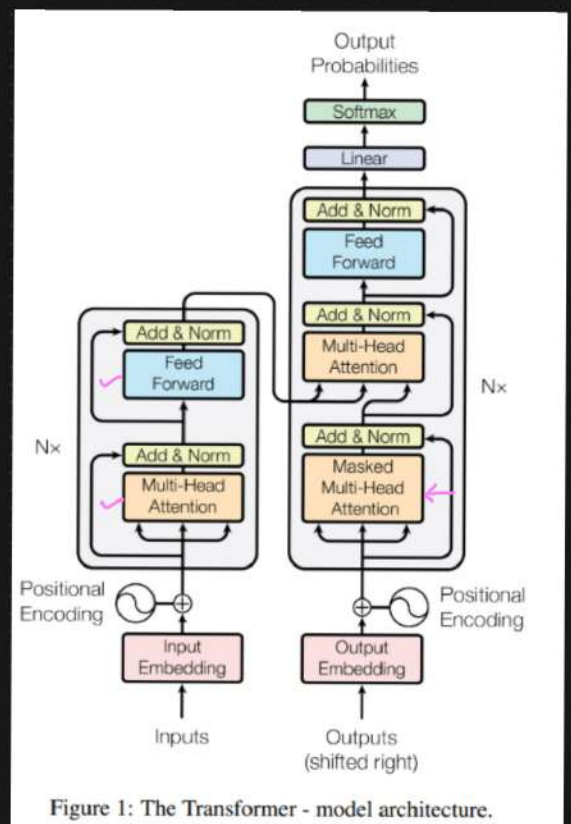
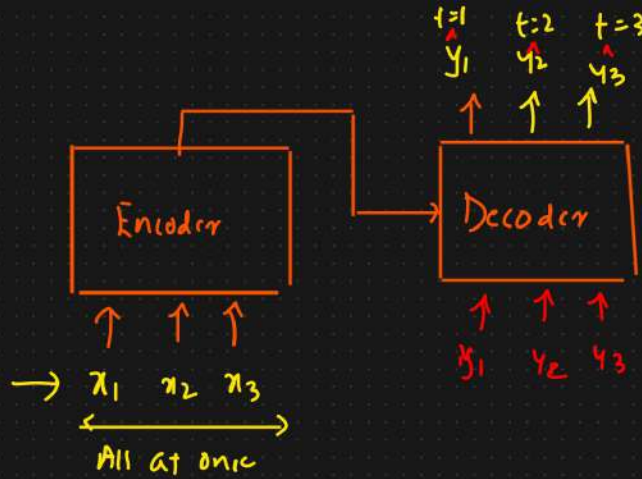


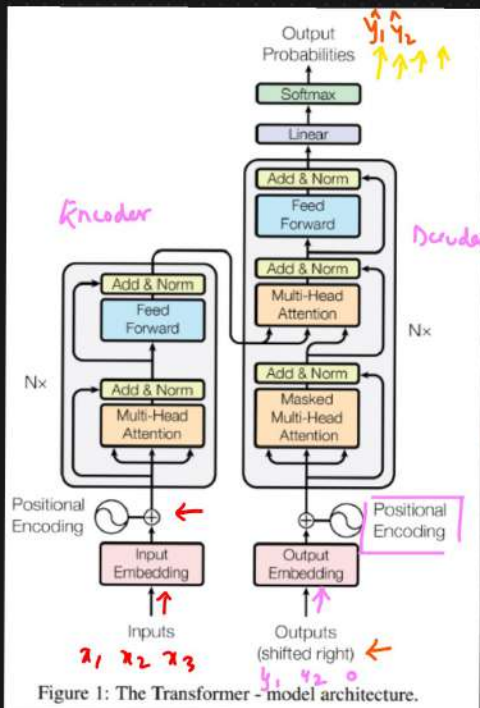
Figure 1: The Transformer - model architecture.



- ① Training Mechanism
- ② Inference Mechanism

* Masked Multi Head Self Attention

- ① I/P Embedding And Positional Embedding ✓ → Zero padding → Sequence length equal
- ② Linear Projection for Q, K, V
- ③ Scaled Dot Product Attention
- ✓ ④ Mask Application } ⇒ Try to understand the imp.
- ⑤ Multi Head Attention
- ⑥ Concatenation And Final Linear Projection
- ⑦ Residual connection And Layer Normalization



Dataset

Eng Hindi

$\langle x_1, x_2, x_3 \rangle$ $\langle y_1, y_2 \rangle$ ←

↓

$\langle y_1, y_2, 0 \rangle$

↓

Zero padding

① Linear Projections Q, K, V

② Scaled Dot Product Attention

③ Mask Application → Look Ahead Mask

→ Padding Mask

Masked
Multi Head
Attention.

IP
[4 5 6 7]

O/P
[1 2 3]

[1 2 3 0]

4 dimension vector

i) Input Embedding and Positional Encoding

Output Embedding [STEP 1]

$$\begin{bmatrix} 0.1, 0.2, 0.3, 0.4 \\ 0.5, 0.6, 0.7, 0.8 \\ 0.9, 1.0, 1.1, 1.2 \\ 0.0, 0.0, 0.0, 0.0 \end{bmatrix} + PE \rightarrow 0$$

Step 2: Linear Projection for Q, K, and V.

$$W_Q = W_K = W_V = I$$

Create query (Q), Key (K) and value (V) vectors

$$\begin{aligned} Q &= \text{Output Embedding} * W_Q = \text{Output Embedding} \\ K &= \text{Output Embedding} * W_K \\ V &= \text{Output Embedding} * W_V \end{aligned}$$

$$Q = K = V = \begin{bmatrix} 0.1, 0.2, 0.3, 0.4 \\ 0.5, 0.6, 0.7, 0.8 \\ 0.9, 1.0, 1.1, 1.2 \\ 0.0, 0.0, 0.0, 0.0 \end{bmatrix} \quad K^T = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix} \quad \begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \\ 0.8 \end{bmatrix} \quad \begin{bmatrix} 0.9 \\ 1.0 \\ 1.1 \\ 1.2 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

③ Scaled Dot Product Attention Calculation

$$\begin{aligned} \text{Scores} &= Q * K^T / \sqrt{d_K} \\ &= Q * K^T / 2. \end{aligned}$$

$$\begin{aligned} &[0.1 * 0.1 + 0.2 * 0.2 + 0.3 * 0.3 + 0.4 * 0.4, \\ &0.1 * 0.5 + 0.2 * 0.6 + 0.3 * 0.7 + 0.4 * 0.8, \\ &0.1 * 0.9 + 0.2 * 1.0 + 0.3 * 1.1 + 0.4 * 1.2, \\ &0.1 * 0 + 0.2 * 0 + 0.3 * 0 + 0.4 * 0] \end{aligned}$$

Scores :

$[0.3, 0.7, 1.1, 0.0]$	[	]
$[0.7, 1.9, 3.1, 0.0]$	[	]
$[1.1, 3.1, 5.1, 0.0]$	[	]
$[0.0, 0.0, 0.0, 0.0]$		

* Masked Application

It helps manage the structure of the sequences being processed
And ensures the models behaves correctly during training And Inferencing

Reasons

① Handling Variable length Sequences with Padding MASK

Purpose

- ① To handle sequences of different length in batch
- ② To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model prediction

Eg: i/p ← Sequence 1 $[1, 2, 3]$ opp
→ 4, 42 0 0 0 0

o/p ← Sequence 2 $[4, 5, 0]$ 0 is the padding token

↑ → Influence the Attention Mechanism

→ 100.

⇓

lead to Incorrect or biased predictions.

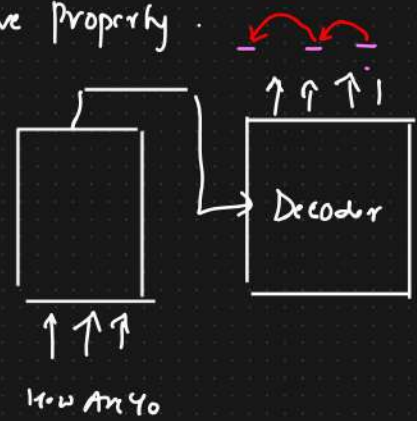
A padding mask ⇒ The tokens Are ignored.

Masking $\left\{ \begin{array}{l} \rightarrow \text{Padding Mask} \\ \rightarrow \text{Look Ahead Mask} \end{array} \right\}$

Padding mask $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$

② Look Ahead Mask → Maintain Auto Regressive Property.

① To ensure that each position in the decoder output sequence can only attend to the previous position, but no future position



② Sequence → Language Modelling, Translation

Eg: $[4, 5, 0] \rightarrow [1, 1, 0]$ → 1D Mask.

Attention Mechanism ← Token 1 attends to token 1, 2

→ $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

Convert 1D to 2D Mask

For each token in the sequence, the mask should indicate which tokens it can attend to.

* Look Ahead Mask → Decoder Output

$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

① Combine Padding And Looking Ahead Mask

Element Wise multiplication of 2 mask

Combine Mask = $\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

④ Mask

Scores :

$$\begin{bmatrix} 0.3, 0.7, 1.1, 0.0 \\ 0.7, 1.9, 3.1, 0.0 \\ 1.1, 3.1, 5.1, 0.0 \\ 0.0, 0.0, 0.0, 0.0 \end{bmatrix}$$

Look Ahead Mask

$$\begin{bmatrix} [1, 0, 0, 0] \\ [1, 1, 0, 0] \\ [1, 1, 1, 0] \\ [1, 1, 1, 1] \end{bmatrix}$$

Padding Mask [extended to 2D Format]

$$\begin{bmatrix} [1, 1, 1, 0] \\ [1, 1, 1, 0] \\ [1, 1, 1, 0] \\ [0, 0, 0, 0] \end{bmatrix}$$

Combined Mask : Look Ahead Mask + Padding Mask

$$\begin{bmatrix} [1 \neq 1, 0 \neq 1, 0 \neq 1, 0 \neq 0] \\ [1 \neq 1, 1 \neq 1, 0 \neq 1, 0 \neq 0] \\ [1 \neq 1, 1 \neq 1, 1 \neq 1, 0 \neq 0] \\ [1 \neq 1, 1 \neq 1, 1 \neq 1, 1 \neq 0] \end{bmatrix} = \begin{bmatrix} [1, 0, 0, 0], \\ [1, 1, 0, 0], \\ [1, 1, 1, 0], \\ [1, 1, 1, 0] \end{bmatrix} \Rightarrow \begin{bmatrix} [1, -\infty, -\infty, -\infty], \\ [1, 1, -\infty, -\infty], \\ [1, 1, 1, -\infty], \\ [1, 1, 1, -\infty] \end{bmatrix}$$

Masked Score

$$\begin{bmatrix} [0.3, -\infty, -\infty, -\infty] \\ [0.7, 1.9, -\infty, -\infty] \\ [1.1, 3.1, 5.1, -\infty] \\ [0.0, 0.0, 0.0, -\infty] \end{bmatrix}$$

Zero out the influence when the Softmax is applied.

Attention weight.

* Softmax

$$\begin{aligned}\text{Softmax Score} &= \text{Softmax}(\text{Masked Scores}) \\ &= \begin{bmatrix} [1.0, 0.0, 0.0, 0.0], \\ [0.3, 0.7, 0.0, 0.0], \\ [0.1, 0.3, 0.6, 0.0], \\ [1.0, 0.0, 0.0, 0.0] \end{bmatrix}\end{aligned}$$

* Weight Sum of Value

$$\text{Attention O/p} = \text{Softmax Scores} * V.$$

Masking

Masking in the transformer architecture is essential for several reasons. It helps manage the structure of the sequences being processed and ensures the model behaves correctly during training and inference. Here are the key reasons for using masking:

1. Handling Variable-Length Sequences with Padding Mask

Purpose

To handle sequences of different lengths in a batch.

To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model's predictions.

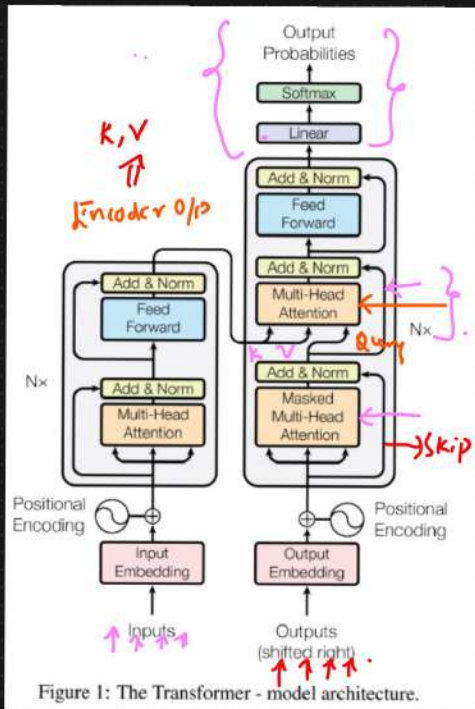
2. Maintaining Autoregressive Property with Look-Ahead Mask

Purpose

To ensure that each position in the decoder output sequence can only attend to previous positions and itself, but not future positions.

This is crucial for sequence generation tasks like language modeling and translation, where the model should not have access to future tokens when predicting the current token.

④ Encoder Decoder Multi Head Attention



- ① Encoder O/p → Set of Attention vector K & V
- ② Masked Multihed → Attention vector Q {Query Vector}

These are to be used by each decoder in its "Encoder - decoder" attention layer



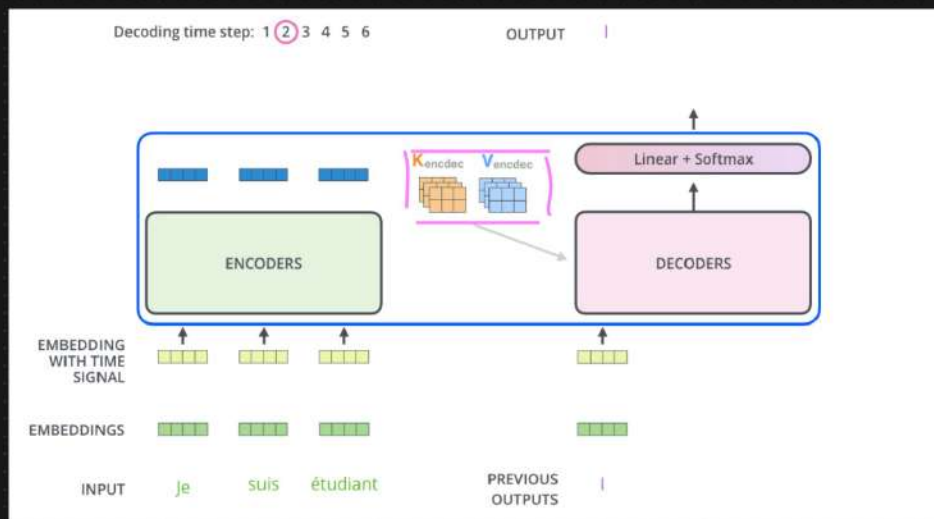
Helps the Decoder to focus on

appropriate places in the i/p Sequence

Data

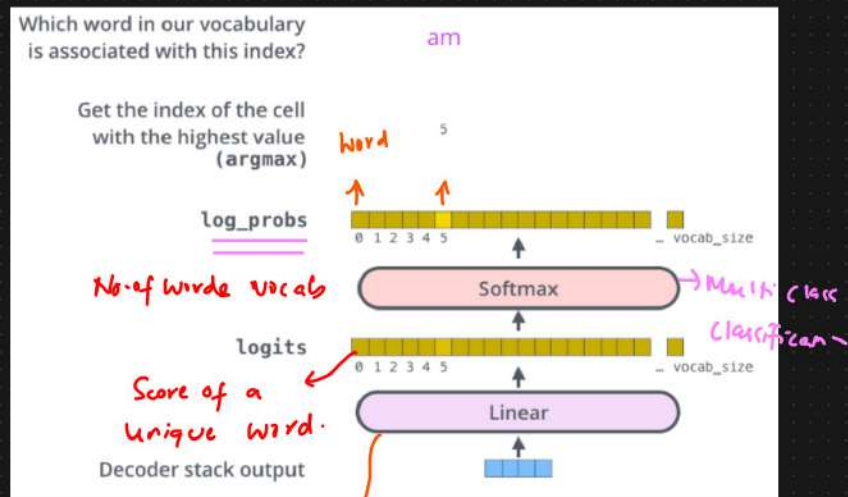
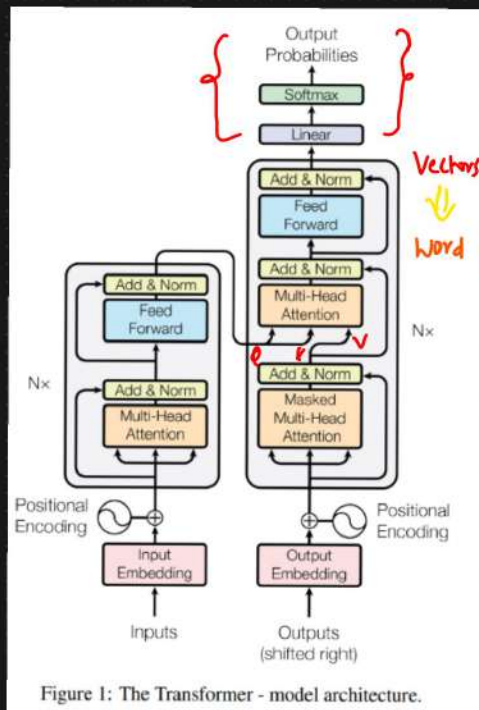
i/p
 $\langle x_1, x_2, x_3 \rangle$

O/p
 $\langle y_1, y_2, y_3 \rangle$



④ The Final Linear And Softmax Layer { Vectors $\rightarrow$ o/p Word }

{ BIOES } $\Rightarrow$ Transformers



linear $\Rightarrow$ The linear layer is a simple fully connected neural net that projects the vector produced by the stack of Decoder $\Rightarrow$ logits vector $\Rightarrow$

Model $\Rightarrow$ 10,000 $\Rightarrow$ Vocabulary $\Rightarrow$ logits vector = 10000 cells wide

② Softmax layer turns those scores into probabilities (all add up to 1.0).

The cell with the highest probability is chosen, and the word associated with it is produced as the o/p. $\Rightarrow$ time stamp.

Recap of Training

Output Vocabulary						
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

ONE ←

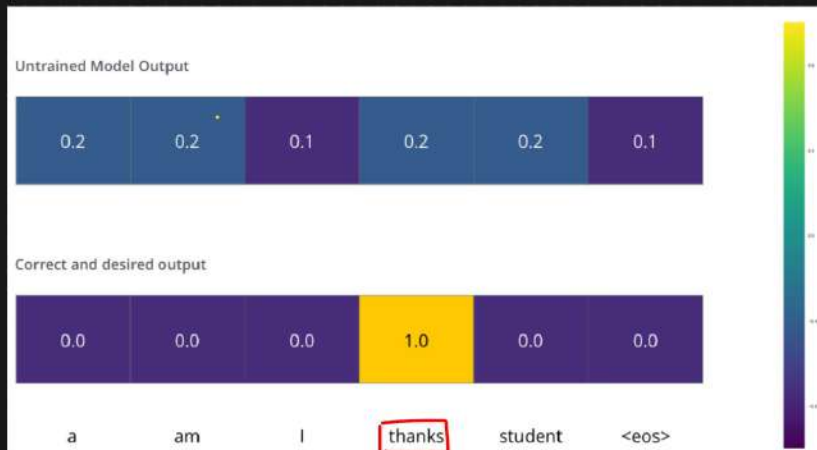
Output Vocabulary						
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"

0.0	1.0	0.0	0.0	0.0	0.0
-----	-----	-----	-----	-----	-----

Merci → Thanks

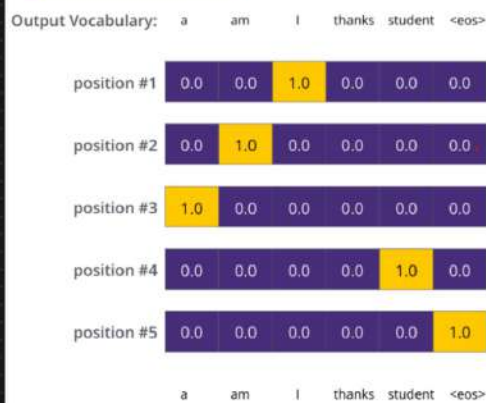
I am a student <eos>



⇒ Loss function ↓↓

Back Propagation

Target Model Outputs

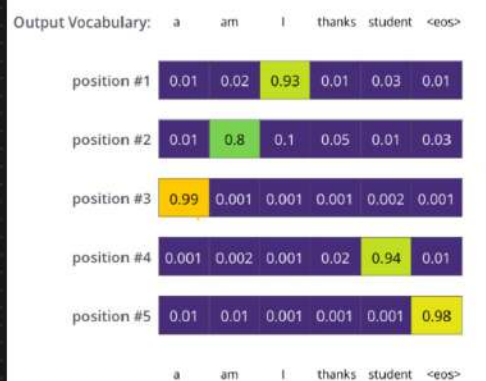


6 6 I, am, I, student

O/p
I am a student
↓ ↓ ↓ ↓
ONE ONE ONE ONE

Loss

Trained Model Outputs



←